

Build Watch — сопроводительная документация

Автоматизированный сервис поиска нарушений на строительных площадках Москвы. Прототип по техническому заданию Департамента градостроительной политики города Москвы, 2026.

Документ соответствует разделу 5 ТЗ: модель данных, организация кода, условия и ограничения, инструкции по сборке и запуску.

1. Что делает система

На вход поступают два независимых потока: снимки с камер видеонаблюдения и календарный план строительно-монтажных работ. Система распознаёт на снимках строительную технику, сводит найденное к суткам, сопоставляет с плановым этапом по методике «этап работ → необходимая техника» и формирует предупреждения об отклонениях с указанием зоны и снимков-подтверждений.

Ключевая единица сопоставления — день, а не отдельный кадр. Один снимок показывает мгновение, а плановый этап длится неделями: если за сутки на объекте хотя бы раз появился самосвал, звено вывоза грунта считается закрытым.

2. Архитектура

Компонент	Технологии	Назначение
ml/	Ultralytics YOLOv8, OpenCV, NumPy	Инференс по кадру, справочник 12 классов техники, пороги уверенности
backend/	FastAPI, Pydantic	REST API, парсинг плана, методика сопоставления, хранилище
backend/domain/	чистый Python	Бизнес-логика: этапы, правила отклонений, статусы. Не зависит от FastAPI и от модели
frontend/	React 18, Vite, Leaflet	Веб-интерфейс: дашборд города, объект, снимок, разбор площадки

Взаимодействие: браузер обращается к `/api/*`; в разработке запросы проксирует Vite, в Docker — nginx. Backend вызывает `ml.infer` внутри процесса, без отдельного сервиса инференса: для прототипа это убирает лишний сетевой хоп и упрощает запуск на ноутбуке.

Слой `backend/domain` намеренно отделён от веба и от модели. Он принимает готовый словарь «класс техники → количество» и возвращает отчёт за день, поэтому методику можно покрыть тестами без GPU и без загрузки весов — см. `backend/test_rules.py`.

3. Модель данных

Хранилище — JSON-документ `data/store.json` с тремя коллекциями и атомарной записью через временный файл (`backend/store.py`). Для прототипа этого достаточно: данные читаются целиком, объём измеряется сотнями записей, а развёртывание не требует

отдельной СУБД. Переход на PostgreSQL или MongoDB затрагивает только backend/store.py — остальной код работает со словарями.

3.1. Объект строительства (sites)

Поле	Тип	Описание
id	string	Идентификатор (UUID или код сид-объекта)
name	string	Название объекта
address	string	Адрес, округ
lat, lng	float	Координаты для карты
plan	array	Календарный план: строки этапов (см. 3.2)
comment	string	Пояснение бизнес-логики для демонстрации
created_at, updated_at	ISO-8601	Метки времени

3.2. Строка календарного плана

Поле	Тип	Описание
stage_code	string	Код этапа: earthworks, piling, monolith, superstructure
date_from	date	Начало этапа
date_to	date	Окончание этапа
zone	string	Зона или участок работ; попадает в текст отклонения

Источник — CSV или Excel. Парсер (backend/plan_parse.py) принимает русские названия этапов («котлован», «сваи», «монолит», «монтаж») и произвольный регистр заголовков, приводя их к кодам через таблицу синонимов.

3.3. Снимок (photos)

Поле	Тип	Описание
id	string	UUID снимка
site_id	string	Ссылка на объект
captured_at	date	Дата съёмки — связь факта с планом
filename	string	Исходное имя файла
path	string	Путь внутри data/photos/<site_id>/
width, height	int	Размер кадра, нужен для отрисовки рамок
model	string	Путь к весам, которыми получены детекции

Поле	Тип	Описание
detections	array	Результат инференса (см. 3.4)

3.4. Детекция техники

Поле	Тип	Описание
class	string	Код класса из ml/classes.py
confidence	float	Уверенность модели, 0..1
bbox	[x1, y1, x2, y2]	Рамка в пикселях исходного кадра

Хранение пути к весам вместе с детекциями сделано осознанно: после переобучения модели видно, какие снимки разобраны старой версией.

3.5. Отчёт за день и отклонения

Отчёты не хранятся — они вычисляются из плана и снимков при каждом запросе (backend/domain/rules.py, функции analyze_day и build_timeline). Это исключает рассинхронизацию: правки методики или плана сразу меняют выводы по всей истории.

Отчёт за день содержит дату, количество техники по классам, распознанные и плановые этапы, статус и список отклонений. Отклонение:

Поле	Тип	Описание
type	string	missing_required, incomplete_link, unexpected_equipment
severity	string	critical или warning
stage_code, stage_label	string	Этап, к которому относится отклонение
message	string	Готовая формулировка для пользователя
zone	string	Затронутая зона из плана
photo_ids	array	Снимки-подтверждения

3.6. Статусы объекта

Статус дня выводится из набора отклонений, объект получает худший статус среди дней со съёмкой (backend/domain/status.py).

Код	Название	Когда возникает
idle	Простой	Неполное звено: техника есть, работать нечем
warning	Возможное нарушение	По плану этап идёт, маркерной техники нет
info	Информация	Техника другого этапа — смена стадии

Код	Название	Когда возникает
ok	В норме	Маркер этапа и звено на месте
no_data	Нет данных	За период нет снимков

4. Методика «этап → необходимая техника»

Методика задаёт две сущности. Маркер — машина, по которой этап опознаётся однозначно. Звено — набор машин, без которых работа физически не идёт: внутри группы достаточно любой машины, но все группы должны быть закрыты одновременно.

Этап	Маркеры	Звено
Земляные работы / котлован	экскаватор, бульдозер	самосвал или грузовик
Свайный фундамент	сваебой	автокран или кран-манипулятор
Монолитные работы	бетономешалка, бетононасос	бетономешалка + бетононасос
Надземная часть / монтаж	башенный кран, автокран, кран-манипулятор	не требуется

Разделение на маркер и звено даёт ту самую ситуацию из ТЗ: на этапе устройства котлована экскаватор виден, самосвалов нет — работа формально идёт, фактически грунт вывозить некому. Система выдаёт предупреждение о возможном снижении темпа работ с зоной и снимками. Этот сценарий закреплён тестом `test_incomplete_link_like_tz_example`.

Классы `loader`, `dump_truck`, `truck`, `roller` объявлены вспомогательными (SUPPORT_ONLY): сами по себе они не открывают этап, иначе проехавший самосвал переводил бы площадку в «земляные работы».

5. Организация кода

```

backend/
  main.py           эндпоинты FastAPI, схемы ответов
  plan_parse.py     CSV/Excel → строки плана
  store.py          JSON-хранилище, атомарная запись
  seed.py           демо-портфель объектов для первого запуска
  test_rules.py     тесты методики (без модели)
  domain/
    stages.py       методика: маркеры, звенья, приоритеты
    rules.py        отчёт за день, отклонения, таймлайн
    status.py       статусы дня и объекта
  ml/
    classes.py      12 классов, русские названия, цвета, пороги
    infer.py        загрузка весов и инференс по кадру
    train_12.py     дообучение на 12 классов
    runs/detect/ml_v2/ метрики текущей модели
  frontend/src/
    pages/          экраны приложения
    components/     переиспользуемые части, отрисовка рамок
  scripts/
    fetch_weights.py скачивание весов
    eval_model.py    проверочный набор и замер скорости
    merge_train_12.py сборка обучающего датасета

```

Комментарии оставлены там, где решение неочевидно: почему порог детектора ниже порога класса, почему вспомогательная техника не открывает этап, почему оверлей рамок на дашборде обрезается так же, как кадр. Тривиальные операции не комментируются.

API

Метод	Путь	Назначение
GET	/api/health	Статус модели, число классов, признак рабочих весов
GET	/api/classes	Каталог техники с порогами и цветами
GET	/api/stages	Методика «этап → техника»
GET	/api/statuses	Справочник статусов
POST	/api/detect	Детекция по одному кадру, включает inference_ms
POST	/api/demo/analyze	План и пачка снимков → таймлайн, отклонения, performance
GET	/api/dashboard	Сводка по всем объектам города
GET	/api/sites, /api/sites/{id}	Объекты и карточка объекта
GET	/api/sites/{id}/timeline	План и факт по дням
POST	/api/sites/{id}/plan	Загрузка календарного плана
POST	/api/sites/{id}/photos	Загрузка снимка
GET	/api/photos/{id}	Снимок с детекциями и выводом алгоритма
GET	/api/demo/pack	Демо-пакет из репозитория для вкладки «Демо»

Интерактивная схема доступна по адресу <http://127.0.0.1:8000/docs>.

6. Условия и ограничения

6.1. Данные обучения

Модель дообучена из предобученных весов yolov8m.pt на объединённом датасете строительной техники: базовые девять классов плюс расширение из открытого набора Roboflow (roller, truck, crane_manipulator, усиление tower_crane). Сборка датасета воспроизводится скриптом scripts/merge_train_12.py, обучение — ml/train_12.py, прогон выполнялся на GPU в Yandex DataSphere.

Параметр	Значение
Архитектура	YOLOv8m
Классов	12
Разрешение обучения	640
Эпох	100

Параметр	Значение
Batch	16
Precision / Recall	0.821 / 0.832
mAP@50	0.859
mAP@50-95	0.651

Матрицы ошибок, кривые обучения и примеры предсказаний на валидации лежат в `ml/runs/detect/ml_v2/`.

6.2. Распознаваемые типы техники

Экскаватор, бульдозер, погрузчик, самосвал, бетономешалка, бетононасос, башенный кран, автокран, сваебой, каток, кран-манипулятор, грузовик.

Перечень ТЗ (самосвал, экскаватор, каток, кран-манипулятор, бетоносмеситель, бульдозер, грузовик, автокран) покрыт полностью; погрузчик, бетононасос, башенный кран и сваебой добавлены сверх него, поскольку без них не различить монолитные работы и надземную часть.

Порог уверенности задан на класс, а не глобально (`ml/classes.py`). Башенный кран и грузовик модель распознаёт хуже — для них порог ниже; бетономешалку, которую легко спутать с обычным грузовиком, отсекаем строже.

6.3. Ограничения прототипа

- Этапы. Логика сопоставления настроена на четыре этапа: земляные работы, свайный фундамент, монолит, надземная часть. Отделочные и инженерные работы по технике на кадре не опознаются — там нет характерных машин.
- Единица анализа — сутки. Внутрисменные простои система не видит.
- Нет трекинга. Количество техники считается по кадру, поэтому одна машина на пяти кадрах за день не отличается от пяти машин. Для контроля наличия звена этого достаточно, для учёта машино-часов — нет.
- Зона берётся из плана, а не из геометрии кадра: привязка отклонения к участку настолько точна, насколько детализирован календарный план.
- Погодные ограничения. Ночные кадры, сильный дождь, туман и залепленный объектив снижают полноту распознавания. Рекомендации по установке камер приведены в презентации.
- Хранилище. JSON-файл не рассчитан на параллельную запись из нескольких процессов; для продуктивной эксплуатации нужна СУБД.
- Веса в репозитории не хранятся (файл около 50 МБ) — скачиваются скриптом.

7. Сборка, установка и запуск

7.1. Требования

Python 3.10 или новее, Node.js 18 или новее. GPU не требуется: прототип работает на CPU, медиана инференса около 0,5 секунды на кадр. Для Docker-варианта нужен только Docker с плагином Compose.

7.2. Запуск через Docker

```
py -m scripts.fetch_weights
docker compose up --build
```

Интерфейс: <http://127.0.0.1:5173>, API: <http://127.0.0.1:8000/docs>. Веса и рантайм-данные монтируются с хоста, поэтому пересборка образа их не теряет.

7.3. Локальный запуск

Backend:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
py -m scripts.fetch_weights
uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend в отдельном терминале:

```
cd frontend
npm install
npm run dev
```

Открыть <http://127.0.0.1:5173>.

7.4. Проверка работоспособности

```
curl http://127.0.0.1:8000/api/health
```

Ответ `{"status": "ok", "classes": 12, "weights_ok": true, "covered_classes": 12}` означает, что загружены наши веса. Значение `"status": "degraded"` говорит, что веса не найдены и модель работает на предобученном yolov8n — запустите `py -m scripts.fetch_weights`.

7.5. Демонстрационный сценарий

- 1 Открыть вкладку «Демо».
- 2 Нажать «Загрузить демо-данные» — план и пять кадров подставятся из папки Демо_конкурс.
- 3 Нажать «Запустить анализ». Появятся таймлайн план/факт, предупреждения со ссылками на снимки и замеры скорости.

Свои данные загружаются в те же два поля: календарный план в формате CSV или Excel и произвольное число снимков.

7.6. Тесты и проверка модели

```
py -m pytest backend/test_rules.py -q
py -m scripts.eval_model
```

Первая команда проверяет методику сопоставления, вторая собирает проверочный набор с разбросом по освещённости и сезону, измеряет скорость и пишет `docs/отчёт_тестирования.md`.